

Zephyr Channel Sounding API

Step Reports, Specification Mapping and Storage Design

How a procedure is configured, what the public API hands you back for modes 0 to 3, what each field means in Core 6.3 terms, how it maps to the symbols of the companion mode reports, and what you must persist to keep the ranging chain reconstructible

This report is a three-way dictionary. Its left-hand column is the Zephyr public Bluetooth Channel Sounding API — the configuration and procedure calls that set a measurement up, the subevent result structure, the step iterator, and the four mode-specific step data layouts. Its middle column is the Bluetooth Core 6.3 host-interface definition that each field implements, with the units, ranges and sentinel values the specification actually mandates. Its right-hand column is the notation of the companion Mode-0 to Mode-3 and Inline Phase Transfer reports, so that a field arriving from the controller can be traced to the quantity it plays in an estimator. Around that dictionary the report adds the two things a dictionary cannot give: a storage schema that says which fields must be persisted raw and which may be derived later, and a per-mode processing recipe that says what each report is actually *for*. Because a result is only interpretable against the configuration that produced it, the setup path is documented first, with particular attention to the values — the nominal step timings above all — that the Host never requests and is told exactly once. It closes with the gaps and traps found while doing the mapping — among them a bitfield whose Zephyr member names appear to sit opposite the specification's bit numbering, a Mode-1 layout variant the documented structure does not express, a timing parameter the specification's prose promises in an event that does not carry it, and a set of sentinel values that will silently poison an estimator if fed to arithmetic.

API source	Zephyr <code>include/zephyr/bluetooth/cs.h</code> , <code>conn.h</code> , <code>hci_types.h</code> ; <code>samples/bluetooth/channel_sounding</code>
Specification	Core 6.3, Vol. 4, Part E (LE CS Subevent Result); Vol. 6, Parts A and H
Companion reports	<i>Mode-0 Frequency and Timing Calibration</i> , Rev. 1.0; <i>Mode-1 RTT Estimation</i> , Rev. 1.0; <i>Mode-2 Phase-Based Ranging</i> , Rev. 1.0; <i>Mode-3 Combined RTT and PBR</i> , Rev. 1.0; <i>Inline Phase Transfer</i> , Rev. 1.0
Document type	Implementation-oriented technical report
Revision date	6 September 2026

Abstract

Zephyr delivers Channel Sounding results as a subevent header plus an opaque byte buffer of concatenated step records, which the application walks with a parse helper and interprets by casting to one of four mode-specific layouts. That design is faithful to the host interface it wraps, and it pushes three decisions onto the application: which layout applies, which fields are meaningful given the device’s role and configuration, and which values are data rather than sentinels.

This report resolves all three, and prefixes them with the configuration that makes a result interpretable at all. Section 2 documents the setup surface — default settings, capability exchange, configuration creation, procedure parameters and procedure enable — separating what the Host requests from what the two Link Layers negotiate and report back, because several quantities the reduction depends on, the nominal step timings among them, are never requested and are delivered exactly once. Section 3 traces the delivery path from callback to individual step. Sections 4–8 document every field of the subevent header and of the mode-0, mode-1, mode-2 and mode-3 step layouts, each against its Core 6.3 definition, giving units, ranges and unavailable-value encodings. Section 9 collapses the result into a single mapping table from Zephyr field to specification parameter to companion report symbol.

The remainder is design guidance rather than reference. Section 10 proposes a storage schema built on one principle: persist the controller’s raw values and derive nothing irreversibly, because every correction in the companion reports — loop-delay calibration, actuation-error removal, unwrap seeding, first-path detection — is one an implementation will want to revise after the fact. Table 9 names, for each item of per-procedure context, the one callback that is its authoritative source and whether it can be recovered afterwards. Section 11 gives the per-mode recipe: what mode-0 buys the other modes, how the mode-1 timing fields become a round-trip time once the omitted nominal offsets are restored, how a pair of mode-2 phase correction terms becomes a two-way phase, and why a mode-3 step is the only one that lets those two be checked against each other. Section 12 records what the mapping exposed, including a packet-quality bitfield whose member names appear inverted relative to the specification’s bit numbering, a mode-1 variant carrying two additional phase correction terms that the documented structure does not describe, and the sample application’s silence on modes 0 and 3.

Status and sourcing. Zephyr structures and signatures are quoted from `include/zephyr/bluetooth/cs.h` and `conn.h` as shipped in the tree this project builds against, cross-read with the project’s public API documentation and sample code; field order and bitfield allocation should be confirmed against the exact revision in use, and Section 12 flags the specific places where that matters. Specification units, ranges and sentinel encodings are taken from the adopted Core 6.3 text. Storage schema, processing recipes and ingest checks are this report’s own recommendations and are not normative.

Contents

Abstract	i
1 Scope, and the three vocabularies	1
1.1 What this report does not cover	1
2 Configuration and procedure setup	1
2.1 The order is fixed by the Link Layer	1
2.2 Local defaults	2
2.3 Capability exchange: the envelope	2
2.4 Creating a configuration	4
2.5 What the configuration complete event adds	4
2.6 Procedure parameters: a request, not a setting	5
2.7 Procedure enable, and the schedule you actually got	5
2.8 Channel classification	6
2.9 The mode-0 FAE table	6
2.10 Antenna paths	6
2.11 CS Test mode: configuration without a peer	7
2.12 What to latch, and when	7
3 The delivery path	8
3.1 The subevent result	8
3.2 The step iterator	8
3.3 The phase correction term helper	9
4 The subevent header	10
4.1 The two frequency quantities are not the same	10
5 Mode-0 step data	11
6 Mode-1 step data	12
6.1 The omitted nominal offsets	12
6.2 The variant the documented structure does not show	13
7 Mode-2 step data	13
7.1 Forming the two-way phase	14
8 Mode-3 step data	14
9 Master mapping	15
10 What to store	15
10.1 Configuration context, stored once per procedure	16
10.2 Per subevent	16
10.3 Per step	16
10.4 Derived values, stored alongside and never in place of the raw	17
11 How to use each report	17
11.1 Mode-0: establish the scale, then gate on it	17
11.2 Mode-1: a coarse unambiguous range, and an integrity verdict	17
11.3 Mode-2: the reported range	18
11.4 Mode-3: use the pairing	18

12 Gaps and traps found while mapping	18
12.1 The packet-quality bitfield may be named opposite the specification's bit numbering	18
12.2 The mode-1 and mode-3 phase variant is not expressed in the structures	18
12.3 The union does not tell you which member is valid	19
12.4 The reference sample covers only modes 1 and 2	19
12.5 The step timings are not in what you ask for	19
12.6 T_{SW} is named in the specification's prose but is not in the event	19
12.7 Timing capability masks enumerate only the optional values	19
12.8 Requested scheduling is not granted scheduling	19
12.9 Sentinels in summary	20
13 Ingest checks worth asserting	20
14 Conclusions	21
A Enumerated values	21
References	22

1 Scope, and the three vocabularies

Three naming systems describe the same measurements, and confusion between them is the main practical obstacle to writing a correct ranging application.

Table 1: The three vocabularies this report reconciles.

Vocabulary	Example	Where it is authoritative
Zephyr API	<code>measured_freq_offset, tod_toa_reflector, tone_info[]</code>	What your code actually compiles against
Core 6.3 HCI	<code>MeasuredFreqOffset, ToD_ToA_Reflector, Tone_PCT[k]</code>	Units, ranges, sentinel values, when a field is present
Companion reports	$\text{FFO}_{RX}[k]$, B_R , $\text{PCT}_{REFL}(f_k)$	What the quantity <i>does</i> in an estimator

The report is organised so that any field can be looked up in any of the three. Sections 4–8 are ordered by Zephyr structure; Section 9 is a single sorted table across all three; Section 11 is ordered by what you are trying to compute.

1.1 What this report does not cover

The estimator theory behind each quantity is in the companion reports and is referenced rather than repeated. Transport of results between devices — the GATT service the reference sample uses to move a reflector’s step data to an initiator — is an application concern and is not treated here, nor is provisioning, pairing policy or power management. Everything the controller interface itself exposes for Channel Sounding is in scope, configuration included.

2 Configuration and procedure setup

A step report is not self-describing. Nothing in the subevent result says which mode layout applies, how long the nominal step was, how many antenna paths were exchanged, or which channels the procedure was allowed to use — and Section 6 shows that without the nominal step timing the reported timing residuals cannot be converted into intervals at all. All of it is fixed during setup, and most of it is only reported back to the Host once, in a completion callback that fires long before the first result. This section documents that surface: what you request, what the two controllers negotiate between themselves, and what you are told afterwards.

The distinction between those three is the organising idea. The Host does not configure a CS procedure so much as bound it. The Link Layer selects the step timings, the controller may ignore the scheduling recommendations, and several quantities the reduction depends on are chosen by the peer pair and merely reported. An application that stores what it *asked for* rather than what it was *told* will store the wrong numbers.

2.1 The order is fixed by the Link Layer

The setup calls are not independent. The specification requires that a CS Start procedure be initiated “only after the CS Security Start procedure has completed, the CS Capability Exchange procedure has completed or the capabilities are previously known, the CS Configuration procedure has completed or the configuration content is previously known, and either the mode-0 FAE table is previously known or the Channel Sounding Mode-0 FAE Table Request procedure has completed” [Core 6.3, Vol. 6, Part B, Sec. 5.1.26]. Table 2 gives the resulting call order and the callback that gates each transition.

Table 2: The setup sequence. Each row must complete — for the asynchronous ones, the callback must have fired — before the next is issued. Callbacks are members of `struct bt_conn.cb` registered with `BT_CONN_CB_DEFINE`.

#	Call	Completion	What it fixes
1	<code>bt_le_cs_set_default_settings</code>	Synchronous	Local roles, <code>CS_SYNC</code> antenna, max TX power
2	<code>bt_conn_set_security(L2)</code>	<code>security_changed</code>	ACL encryption; prerequisite for CS security
3	<code>bt_le_cs_read_remote_supported_capabilities</code>	<code>le_cs_read_remote_capabilities_complete</code>	The feasible envelope, §2.3
4	<code>bt_le_cs_read_remote_fae_table</code> (conditional)	<code>le_cs_read_remote_fae_table_complete</code>	Per-channel FAE, §2.9
5	<code>bt_le_cs_create_config</code>	<code>le_cs_config_complete</code>	Mode, role, RTT type, PHY, channel map — and the step timings , §2.5
6	<code>bt_le_cs_security_enable</code>	<code>le_cs_security_enable_complete</code>	CS-level security. Resets procedure_counter
7	<code>bt_le_cs_set_procedure_parameters</code>	Synchronous	Requested scheduling, ACI, PHY, SNR control, §2.6
8	<code>bt_le_cs_procedure_enable</code>	<code>le_cs_procedure_enable_complete</code>	The schedule actually granted, §2.7
9	—	<code>le_cs_subevent_data_available</code>	Results, §3

Step 6 is why `procedure_counter` is a usable grouping key. The header field of Table 10 counts procedures *since the CS Security Start procedure*, so it is zeroed at step 6 and is unique only within one security session. A dataset that spans a re-enable must key on the security session as well, or two distinct procedures will collide on counter value.

2.2 Local defaults

`bt_le_cs_set_default_settings` configures the local controller only and takes effect per connection. It is the only place the local roles are enabled; a device that has not enabled the reflector role will not respond to a peer's configuration request for it.

Table 3: `struct bt_le_cs_set_default_settings_param`.

Field	Meaning	Note for reduction
<code>enable_initiator_role</code> , <code>enable_reflector_role</code>	Which CS roles the local controller will accept	Store: the local role decides which union member of every step is valid (§3)
<code>cs_sync_antenna_selection</code>	<code>enum bt_le_cs_sync_antenna_selection_opt</code> : antenna 1–4, repetitive, or no recommendation	Determines which antenna produced a given <code>packet_antenna</code> ; under REPETITIVE it cycles
<code>max_tx_power</code>	EIRP ceiling for all CS transmissions, –127 to 20 dBm	Bounds the link budget; the granted value appears in <code>selected_tx_power</code> (Table 8)

2.3 Capability exchange: the envelope

`bt_le_cs_read_remote_supported_capabilities` triggers the exchange; the result arrives as `struct bt_conn_le_cs_capabilities` in the completion callback. The local side is readable synchronously through `bt_le_cs_read_local_supported_capabilities`. Table 4 lists the members that constrain a ranging design rather than the full structure.

Table 4: Capability fields that bound the configuration. `_v2` variants of the read and cached-write calls carry the Core 6.3 additions (IPT, T_{SW} during IPT).

Field	Meaning	What it constrains
<code>num_config_supported</code>	Number of simultaneous CS configurations	How many <code>config_id</code> values may coexist
<code>num_antennas_supported</code> , <code>max_antenna_paths_supported</code>	Antenna elements and paths	The admissible ACI, hence N_{AP} and the tone array length $N_{AP} + 1$
<code>initiator_supported</code> , <code>reflector_supported</code> , <code>mode_3_supported</code>	Role and mode-3 support	Whether the mode-3 pairing of §8 is available at all
<code>rtt_aa_only_precision</code> , <code>rtt_sounding_precision</code> , <code>rtt_random_payload_precision</code>	Not supported / 10 ns / 150 ns	Choice of <code>rtt_type</code> ; 150 ns time-of-flight accuracy is 22.5 m, so AA-only at 150 ns is a gate, not a ranging mode
<code>rtt_aa_only_n</code> , <code>rtt_sounding_n</code> , <code>rtt_random_payload_n</code>	Steps needed to reach that accuracy	A floor on <code>max_main_mode_steps</code> and on the averaging depth of the Mode-1 report
<code>phase_based_nadm_sounding_supported</code> , <code>...random_supported</code>	Phase-based NADM availability	Whether <code>packet_nadm</code> carries a phase-based verdict or only 0xFF
<code>cs_sync_2m_phy_supported</code> , <code>cs_sync_2m_2bt_phy_supported</code>	CS_SYNC PHY options	T_{SY} , and β_{rms} in the Mode-1 bound
<code>cs_without_fae_supported</code>	Peer guarantees zero FAE as reflector	If true, skip the FAE read; calling it returns an error event (§2.9)
<code>pbr_from_rtt_sounding_seq_supported</code>	Phase-based ranging from the RTT sounding sequence	Together with a sounding <code>rtt_type</code> , this is the precondition for the mode-1 <code>Packet_PCT</code> variant of §6.2
<code>cs_ipt_reflector</code>	Peer supports IPT in the reflector	Precondition for setting <code>cs_enhancements_1</code> bit 0
<code>t_ip1_times_supported</code> , <code>t_ip2_times_supported</code> , <code>t_fcs_times_supported</code> , <code>t_pm_times_supported</code>	Bitmasks of the <i>optional shorter</i> durations	Bound what the Link Layer may select; see the caution below
<code>t_sw_time</code> , <code>t_sw_ipt_time_supported</code>	Antenna switch period, normally and during IPT	The only source of T_{SW} , which the configuration event does not report (§2.5)
<code>tx_snr_capability</code>	Supported SNR levels in RTT packets	Admissible <code>snr_control_*</code> values

An all-zero timing capability mask does not mean “unsupported”. The masks enumerate the optional durations only. For `t_ip1_times_supported` and `t_ip2_times_supported` the Zephyr comment lists bits 0–6 as 10, 20, 30, 40, 50, 60 and 80 μ s; the mandatory value is index 7, 145 μ s, and every conformant device has it [Core 6.3, Vol. 6, Part H, Tables 4.5, 4.9]. Likewise `t_fcs_times_supported` covers 15 to 120 μ s while the mandatory T_{FCS} is 150 μ s (index 9), and `t_pm_times_supported` covers 10 and 20 μ s while the mandatory T_{PM} is 40 μ s (index 2). A peer reporting zero in all four is not incapable of Channel Sounding; it simply offers nothing faster than the mandatory timings. Do not use these masks to compute the timings — read them from the configuration event instead.

`bt_le_cs_write_cached_remote_supported_capabilities` (and its `_v2` form) writes a previously obtained capability set into the local controller, which lets a reconnecting device skip step 3 of Table 2. For a measurement campaign this is a convenience with a trap attached: the cached values are then not observed on this connection, so they must still be recorded in the per-procedure context from wherever they were cached.

2.4 Creating a configuration

`bt_le_cs_create_config` takes `struct bt_le_cs_create_config_params` and a context argument, `BT_LE_CS_CREATE_CONFIG_CONTEXT_LOCAL_ONLY` or `..._LOCAL_AND_REMOTE`. Only the latter runs the Channel Sounding Configuration procedure over the air; the former writes the local controller alone and is for the case where the peer was configured by other means.

Table 5: `struct bt_le_cs_create_config_params`. Ranges from [Core 6.3, Vol. 4, Part E, LE CS Create Config].

Field	Value	Consequence for the result
<code>id</code>	Configuration identifier, 0–3	Appears as <code>header.config_id</code>
<code>mode</code>	<code>enum bt_conn_le_cs_mode</code> , packing main mode in bits 0–1 and sub mode in bits 4–5	Decides which step layouts appear. Recover the parts with <code>BT_CONN_LE_CS_MODE_MAIN_MODE_PART</code> and <code>..._SUB_MODE_PART</code>
<code>min_main_mode_steps</code> , <code>max_main_mode_steps</code>	1–255 each: main mode steps between submode steps	The interleaving pattern; a submode step is not announced, it is identified by <code>step->mode</code>
<code>main_mode_repetition</code>	0–3	Steps repeated from the previous subevent: repeated steps revisit an earlier channel and are not independent samples
<code>mode_0_steps</code>	1–3	How many mode-0 steps open each subevent; the averaging depth available for the Mode-0 frequency estimate
<code>role</code>	Initiator or reflector	The local role. Fixes the valid union member for every step
<code>rtt_type</code>	AA-only, 32/96-bit sounding, 32/64/96/128-bit random	Decides the mode-1 and mode-3 step length , and with <code>pbr_from_rtt_sounding_seq_supported</code> whether the <code>Packet_PCT</code> variant is present (§6.2)
<code>cs_sync_phy</code> <code>channel_map[10]</code>	1M, 2M or 2M-2BT 79-bit map. Channels 0, 1, 23, 24, 25, 77, 78 disallowed, 79 reserved, at least 15 enabled	T_{SY} , and β_{rms} in the Mode-1 bound The aperture. The Mode-2 report shows the largest gap sets the unwrap limit
<code>channel_map_repetition</code>	1–255 cycles of the map for non-mode-0 steps	With the map size, the total non-mode-0 step budget of a procedure
<code>channel_selection_type</code>	Algorithm #3b or #3c	#3c requires <code>chsel_alg_3c_supported</code> on both sides
<code>ch3c_shape</code> , <code>ch3c_jump</code>	Hat or X; jump 2–8	Meaningful only under #3c; they shape the channel visiting order
<code>cs_enhancements_1</code>	Bit 0: IPT enabled in the CS reflector. Other bits reserved	Whether the reflector's Q is meaningful ; see the IPT companion report

Two helpers exist for the channel map. `BT_LE_CS_CHANNEL_BIT_SET_VAL(chmap, bit, val)` and `BT_LE_CS_CHANNEL_BIT_GET` address individual channels, and `bt_le_cs_set_valid_chmap_bits` fills a map with every permitted channel. Neither enforces the “at least 15 enabled” rule; violating it is not rejected at `bt_le_cs_create_config` but surfaces later as an *Insufficient Channels* (0x48) error on `bt_le_cs_set_procedure_parameters` [Core 6.3, Vol. 4, Part E].

`bt_le_cs_remove_config` tears a configuration down and fires `le_cs_config_removed`. It also invalidates the procedure parameters set for that `config_id`, so a configuration that is removed and recreated must have step 7 of Table 2 repeated.

2.5 What the configuration complete event adds

The callback delivers `struct bt_conn_le_cs_config`, which echoes the request and adds four fields that were never part of it. They are the nominal step timings, and they are chosen by the two Link Layers — each “shall be selected from one of the valid values ... that is greater than or

equal to the value contained in the peer’s `LL_CS_CAPABILITIES`” [Core 6.3, Vol. 6, Part B, Table 5.2] — not by the Host.

Table 6: Fields present in `bt_conn_le_cs_config` but absent from `bt_le_cs_create_config_params`. Permitted values from [Core 6.3, Vol. 6, Part H, Tables 4.3, 4.5, 4.8, 4.9].

Field	Permitted values (μs)	Role in the reduction
<code>t_ip1_time_us</code>	10, 20, 30, 40, 50, 60, 80, 145	T_{IP1} of Eq. (2). Interlude between the mode-0 and mode-1 <code>CS_SYNC</code> packets
<code>t_ip2_time_us</code>	10, 20, 30, 40, 50, 60, 80, 145	T_{IP2} . Interlude between the CS tones; enters the mode-2 nominal
<code>t_fcs_time_us</code>	15, 20, 30, 40, 50, 60, 80, 100, 120, 150	T_{FCS} . The frequency-hop period separating steps
<code>t_pm_time_us</code>	10, 20, 40	T_{PM} . Phase measurement period per tone slot; with T_{SW} and N_{AP} it gives the tone duration $(T_{SW} + T_{PM})(N_{AP} + 1)$

T_{SW} is not in this event, although the specification’s prose says it is. The description of the LE CS Config Complete event states that “the `T_IP1_Time`, `T_IP2_Time`, `T_FCS_Time`, `T_PM_Time`, and `T_SW_Time` parameters indicate the time durations used in CS steps” [Core 6.3, Vol. 4, Part E, Sec. 7.7.65.42], but `T_SW_Time` does not appear in that event’s parameter list, and Zephyr’s `bt_conn_le_cs_config` matches the parameter list rather than the prose. T_{SW} must therefore be taken from the capability exchange — `t_sw_time` of each side’s `bt_conn_le_cs_capabilities`, or `t_sw_ipt_time_supported` when IPT is enabled — and it is needed, because the mode-2 tone duration is $(T_{SW} + T_{PM})(N_{AP} + 1)$ and the antenna-path assignment of §7 advances one T_{SW} slot at a time.

T_{RD} completes the set and is not configurable at all: it is a fixed $5\mu\text{s}$ ramp-down window [Core 6.3, Vol. 6, Part H, Sec. 4.3.1]. With `t_ip1_time_us` from Table 6, T_{SY} from `cs_sync_phy`, and $T_{RD} = 5\mu\text{s}$, every term of the nominal offset in Eq. (2) is now accounted for, and the mode-1 timing residual can be turned into an interval.

This is the answer to the open item in Table 17. That table lists the nominal timings as coming from “configuration” and notes that none of it arrives in the subevent result. Precisely: they arrive once, in `le_cs_config_complete`, and they are not what you asked for — you never asked. Latch `bt_conn_le_cs_config` in that callback and store it with the procedure. If it is discarded, the procedures collected under it are not reducible, and no later query can recover the value because the negotiation left no other trace.

2.6 Procedure parameters: a request, not a setting

`bt_le_cs_set_procedure_parameters` bounds the scheduling. The specification is explicit that several of its fields are advisory: “the values for `Min_Procedure_Interval`, `Max_Procedure_Interval`, `Min_Subevent_Len`, and `Max_Subevent_Len` are recommendations to the Controller which it may ignore” [Core 6.3, Vol. 4, Part E, Sec. 7.8.140].

2.7 Procedure enable, and the schedule you actually got

`bt_le_cs_procedure_enable` carries only a `config_id` and an `enable` member of type `enum bt_conn_le_cs_procedure_enable_state`. The completion callback is where the controller reports the resolved schedule, and this structure is the one most worth storing verbatim: every advisory field of Table 7 is answered here with the value that will actually be used.

Table 7: struct `bt_le_cs_set_procedure_parameters_param`.

Field	Value and unit	Note
<code>config_id</code>	0–3	Must name an existing configuration
<code>max_procedure_len</code>	$N \times 0.625$ ms, $N = 1$ to 65535	Duration ceiling per procedure. Binding, not advisory
<code>min_procedure_interval</code> , <code>max_procedure_interval</code>	Connection events between procedures, 1 to 65535	Advisory. Ignored entirely when <code>max_procedure_count</code> is 1
<code>max_procedure_count</code>	0 = until disabled, else 1 to 65535	The campaign length
<code>min_subevent_len</code> , <code>max_subevent_len</code>	Microseconds, 1250 to 3 999 999	Advisory. Together with the channel map they determine how many subevents a procedure needs
<code>tone_antenna_config_selection</code>	ACI 0–7	Requests N_{AP} , hence the tone array length. Confirmed in Table 8, and reported per subevent as <code>num_antenna_paths</code>
<code>phy</code>	1M, 2M, Coded S8, Coded S2	The PHY for the power-delta reference, distinct from <code>cs_sync_phy</code>
<code>tx_power_delta</code>	Signed dB; 0x80 = no recommendation	Recommended offset between the peer’s tone/ <code>CS_SYNC</code> power and its PHY power
<code>preferred_peer_antenna</code>	Bitmask of <code>BT_LE_CS_PROCEDURE_PREFERRED_PEER_</code> <code>ANTENNA_1..4</code>	The number of bits set shall be at least the antenna element count implied by the ACI
<code>snr_control_initiator</code> , <code>snr_control_reflector</code>	18/21/24/27/30 dB, or not used	Bounded by <code>tx_snr_capability</code> ; affects the NADM and quality statistics

2.8 Channel classification

`bt_le_cs_set_channel_classification` takes the same 10-octet bitfield shape as the configuration’s channel map but is a different mechanism: it is not per configuration, it reports the local device’s own view of which channels are usable, and it may be issued while procedures are running. The effective aperture is the intersection of the configured map with the classification on both sides, which means **the aperture can narrow between procedures without any configuration event firing**. Two consequences follow for a measurement campaign: the classification in force is part of the per-procedure context, and a procedure that aborts with `procedure_abort_reason` = “too few channels” is usually explained here rather than by the configuration.

2.9 The mode-0 FAE table

`bt_le_cs_read_remote_fae_table` requests the peer’s per-channel mode-0 Frequency Actuation Error table, delivered as struct `bt_conn_le_cs_fae_table` holding a pointer to 72 signed octets — one per allowed CS channel. This is the FAE correction of the Mode-0 report, and it is the one piece of per-procedure context that is neither a configuration choice nor a result field.

If the peer set `cs_without_fae_supported`, it guarantees zero actuation error as a reflector and the read is unnecessary; calling it anyway completes the callback with an error status rather than a table, which is easily mistaken for a failure. `bt_le_cs_write_cached_remote_fae_table` injects a stored table and, like the cached capabilities, moves a value out of the observable connection and into application state that must be recorded separately.

2.10 Antenna paths

The helper

```
int bt_le_cs_get_antenna_path(uint8_t n_ap, uint8_t antenna_path_permutation_index, uint8_t
tone_index)
```

implements the permutation of [Core 6.3, Vol. 6, Part H, Sec. 4.7.5] and returns the antenna path,

Table 8: struct `bt_conn_le_cs_procedure_enable_complete`: the granted schedule.

Field	Value and unit	Why it matters
<code>config_id, state</code>	Identifier; enabled or disabled	Ties the schedule to the configuration
<code>tone_antenna_config_selection</code>	ACI 0–7, granted	The authoritative N_{AP} , and with it the antenna-path geometry of §7. Cross-check against <code>header.num_antenna_paths</code>
<code>selected_tx_power</code>	dBm, –127 to 20; 0x7F unavailable	Link budget. A sentinel like the header’s <code>reference_power_level</code> and must be tested the same way
<code>subevent_len</code>	Microseconds, 1250 to 4 s	The granted subevent duration, not the requested range
<code>subevents_per_event</code>	1 to 32	How many subevents share one ACL anchor
<code>subevent_interval</code>	$N \times 0.625$ ms	Spacing of subevents within an event
<code>event_interval</code>	ACL connection events	Spacing of CS event anchors
<code>procedure_interval</code>	ACL connection events	The measurement cadence. The time base against which target motion is judged
<code>procedure_count</code>	0 = until disabled	How many procedures to expect; a completeness check on the dataset
<code>max_procedure_len</code>	$N \times 0.625$ ms	The granted ceiling

0–3, that carried a given tone. It is the bridge between the mode-2 step’s `antenna_permutation_index` and its `tone_info[]` array: the array is in tone order, per-path calibration is in path order, and the two coincide only for $N_{AP} = 1$. Passing `tone_index` equal to `n_ap` addresses the extension slot. Any per-path correction applied without this mapping is silently applied to the wrong path whenever the permutation index is non-zero.

2.11 CS Test mode: configuration without a peer

`bt_le_cs_start_test` takes struct `bt_le_cs_test_param` and needs no connection, no capability exchange and no configuration procedure. It requires `CONFIG_BT_CHANNEL_SOUNDING_TEST`, and results arrive through `bt_le_cs_test_cb_register` on a callback that takes a `bt_conn_le_cs_subevent_result` without a `bt_conn`. Everything else in this report applies to those results unchanged.

What changes is the provenance of the context. The test parameters restate the configuration fields of Table 5 and then set directly what the connected path negotiates: `t_ip1_time`, `t_ip2_time`, `t_fcs_time`, `t_pm_time` and — uniquely — `t_sw_time`, alongside `tone_antenna_config_selection`, `subevent_len`, `subevent_interval`, `max_num_subevents`, `transmit_power_level` and the SNR controls. It also exposes `drbg_nonce` and an `override_config` bitmask with per-index override structures that pin the channel sequence, the tone extension pattern, the antenna permutation, the sounding-sequence marker positions and the `CS_SYNC` payload.

Test mode is the right place to validate a reduction pipeline, for exactly the reason this section exists: the nominal timings are known by construction rather than reported, the channel sequence can be pinned, and `drbg_nonce` makes the tone extension pattern reproducible. A pipeline whose loop-delay calibration or unwrap seeding is wrong will show it against a configuration you chose, before a negotiated one can hide it.

2.12 What to latch, and when

Table 9 collapses the section into the storage rule: each item of per-procedure context, the single callback or call that is its authoritative source, and the moment after which it can no longer be recovered.

Table 9: Authoritative source for each per-procedure context item of Table 17.

Context item	Authoritative source	Recoverable later?
Local role	<code>create_config_params.role</code> , confirmed in <code>bt_conn_le_cs_config.role</code>	Yes, from the configuration
Main/sub mode, RTT type, CS_SYNC PHY	<code>bt_conn_le_cs_config</code>	Yes
T_{IP1} , T_{IP2} , T_{FCS} , T_{PM}	<code>bt_conn_le_cs_config</code> , §2.5	No — negotiated, reported once
T_{SW}	<code>bt_conn_le_cs_capabilities.t_sw_time</code> ; <code>t_sw_ipt_time_supported</code> under IPT	No — and not in the config event
T_{RD}	Fixed $5\mu s$ [Core 6.3, Vol. 6, Part H, Sec. 4.3.1]	Constant
T_{SY}	Derived from <code>cs_sync_phy</code>	Yes
N_{AP} , antenna configuration	<code>procedure_enable_complete.tone_antenna_config_selection</code> ; per subevent in <code>header.num_antenna_paths</code>	Partly — the header gives N_{AP} but not the configuration
Channel map	<code>bt_conn_le_cs_config.channel_map</code>	Yes
Channel classification in force	<code>bt_le_cs_set_channel_classification</code> , your own state	No — may change mid-campaign
IPT enabled	<code>bt_conn_le_cs_config.cs_enhancements_1</code> bit 0	Yes
Scheduling cadence	<code>procedure_enable_complete</code> , Table 8	No — granted values differ from requested
FAE table	<code>le_cs_read_remote_fae_table_complete</code>	No
Security session	Latched at <code>le_cs_security_enable_complete</code>	No — disambiguates <code>procedure_counter</code>

3 The delivery path

Results reach the application as a subevent at a time. Each subevent carries a fixed header and an opaque buffer holding the concatenated step records, which a helper walks one step at a time.

3.1 The subevent result

```

struct bt_conn_le_cs_subevent_result {
    struct {
        uint8_t config_id;
        uint16_t start_acl_conn_event;
        uint16_t procedure_counter;
        uint16_t frequency_compensation;
        int8_t reference_power_level;
        enum bt_conn_le_cs_procedure_done_status procedure_done_status;
        enum bt_conn_le_cs_subevent_done_status subevent_done_status;
        enum bt_conn_le_cs_procedure_abort_reason procedure_abort_reason;
        enum bt_conn_le_cs_subevent_abort_reason subevent_abort_reason;
        uint8_t num_antenna_paths;
        uint8_t num_steps_reported;
        uint8_t abort_step;
    } header;
    /* NULL if num_steps_reported is 0 */
    struct net_buf_simple *step_data_buf;
};

```

3.2 The step iterator

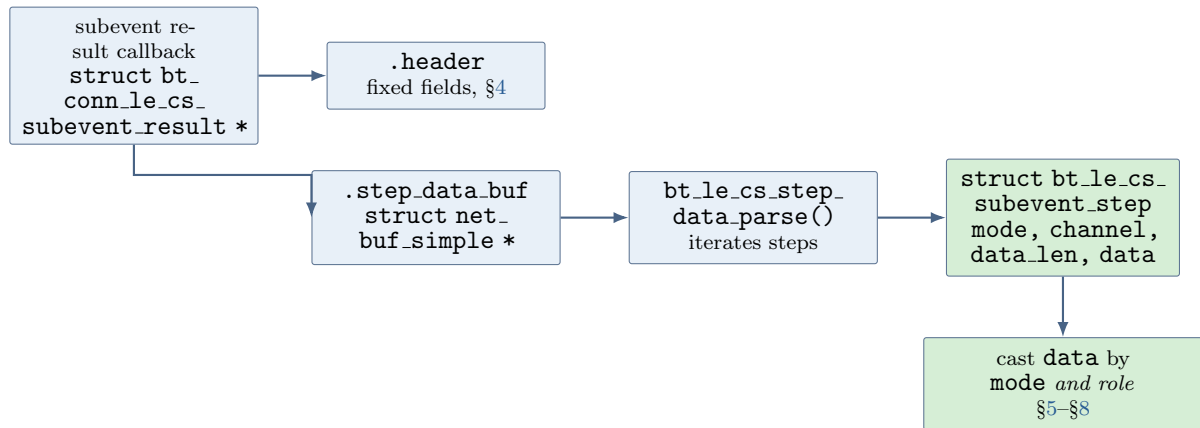


Figure 1: From callback to a single interpreted step. The header is typed; the step payload is not — its layout is determined by the step mode *and* by the local device’s role, neither of which is carried inside the payload itself.

```

void bt_le_cs_step_data_parse(struct net_buf_simple *step_data_buf,
                             bool (*func)(struct bt_le_cs_subevent_step *step,
                                           void *user_data),
                             void *user_data);

struct bt_le_cs_subevent_step {
    uint8_t mode;
    uint8_t channel;
    uint8_t data_len;
    const uint8_t *data;
};
  
```

The callback returns true to continue iterating and false to stop. Note what struct bt_le_cs_subevent_step contains and, more importantly, what it does not: there is no step index, no role, and no indication of which layout variant data holds. The step index must be maintained by the caller as an iteration counter — and it matters, because header.abort_step is expressed in it.

3.3 The phase correction term helper

```

struct bt_le_cs_iq_sample {
    int16_t i;
    int16_t q;
};

/* Sign-extends the 12-bit I and Q halves of a 3-octet phase correction term. */
struct bt_le_cs_iq_sample bt_le_cs_parse_pct(const uint8_t pct[3]);
  
```

Use the helper; do not unpack the term by hand. The phase correction term packs two *signed* 12-bit values into three octets [Core 6.3, Vol. 4, Part E]. A hand-rolled unpack that masks without sign-extending yields values in [0, 4095] instead of [−2048, 2047], which silently folds the negative half-plane onto the positive one. Every phase in the lower half of the complex plane then comes out wrong, and the resulting range is wrong without being obviously so.

Table 10: Subevent header fields. Ranges, units and unavailable encodings are from [Core 6.3, Vol. 4, Part E, LE CS Subevent Result].

Zephyr field	Specification meaning	Use, and companion-report symbol
<code>config_id</code>	CS configuration identifier, 0–3. Zero and meaningless for CS Test results	Store: identifies which configuration the geometry belongs to
<code>start_acl_conn_event</code>	Starting ACL connection event counter	Store: ties the subevent to the connection timeline
<code>procedure_counter</code>	CS procedure count since the CS Security Start procedure	Primary grouping key. All steps of one procedure share it
<code>frequency_compensation</code>	Frequency_Compensation: 15-bit signed, units 0.01 ppm, range ± 100 ppm. <code>0xC000</code> = unavailable <i>or</i> role is not initiator	The subevent frequency reference: FFO_{RX}^* of the Mode-0 report. Divide by 100 for ppm; this is $\varepsilon \times 10^6$
<code>reference_power_level</code>	Reference_Power_Level: dBm, range -127 to 20 ; <code>0x7F</code> not applicable	RPL of the Mode-2 report. Needed to convert PCT magnitude to dBm, Eq. (5)
<code>procedure_done_status</code>	Complete / partial / aborted	Gate: only combine procedures whose status you trust
<code>subevent_done_status</code>	Complete / aborted	Gate
<code>procedure_abort_reason</code>	No abort, requested, too few channels, channel-map instant passed, unspecified	Diagnostic; “too few channels” directly explains a degraded aperture
<code>subevent_abort_reason</code>	No abort, requested, no <code>CS_SYNC</code> received, scheduling conflict, unspecified	Diagnostic
<code>num_antenna_paths</code>	Num_Antenna_Paths: 0 means no phase measurement in the step; otherwise 1–4	N_{AP} . Determines the tone array length , which is $N_{AP} + 1$
<code>num_steps_reported</code>	Number of CS steps in the subevent	Loop bound; cross-check against the iterator count
<code>abort_step</code>	Step number at which the subevent aborted; 255 when complete	Steps $0 \dots \text{abort_step} - 1$ are valid; the rest are not

4 The subevent header

Two header fields are sentinels, not numbers. `frequency_compensation` is `0xC000` when unavailable *or* when the local device is the reflector, and `reference_power_level` is `0x7F` when not applicable. Both are in range for their declared types and neither is flagged separately. Testing for them before use is not defensive programming; it is required, because `0xC000` interpreted as a signed 0.01 ppm value is -163.84 ppm, which exceeds the specification’s own ± 50 ppm bound on a Mode-0 step and would be applied as a frequency correction fifteen hundred times larger than a realistic one.

4.1 The two frequency quantities are not the same

The header’s `frequency_compensation` and the mode-0 step’s `measured_freq_offset` share units and encoding and are easily conflated. They are different quantities in the Mode-0 report’s terms.

Table 11: The two fractional frequency quantities.

	Where	Scope	Companion symbol
<code>measured_freq_offset</code>	Mode-0 step, initiator only	One mode-0 step, one channel	$\text{FFO}_{RX}[k]$
<code>frequency_compensation</code>	Subevent header	The subevent; what the controller actually applied	FFO_{RX}^*

Store both. The per-step values let you assess the Mode-0 report’s combining rule and detect the channel-correlated spread that indicates a stale actuation-error table; the subevent value is

what the controller used, and is therefore the one that already influenced the mode-1, mode-2 and mode-3 steps you are about to process.

5 Mode-0 step data

Mode-0 is the only mode whose layout differs by role in the number of fields, because only the initiator measures a frequency offset [Core 6.3, Vol. 6, Part H, Sec. 4.3.1].

```
struct bt_hci_le_cs_step_data_mode_0_initiator {
    uint8_t packet_quality_bit_errors : 4;
    uint8_t packet_quality_aa_check : 4;
    uint8_t packet_rssi;
    uint8_t packet_antenna;
    uint16_t measured_freq_offset;
} __packed;

struct bt_hci_le_cs_step_data_mode_0_reflector {
    uint8_t packet_quality_bit_errors : 4;
    uint8_t packet_quality_aa_check : 4;
    uint8_t packet_rssi;
    uint8_t packet_antenna;
} __packed;
```

Table 12: Mode-0 step fields.

Zephyr field	Specification meaning	Use, and companion symbol
packet_quality_aa_check	Packet.Quality bits 0–3: 0x0 access address matched exactly, 0x1 one or more bit errors, 0x2 access address not found	Primary gate. Anything but 0x0 means the timing reference for the subevent is suspect
packet_quality_bit_errors	Packet.Quality bits 4–7: payload bit-error count when the RTT type carries a random or sounding sequence; 0 may mean zero errors <i>or</i> no report; 15 means 15 or more	Mode-0 packets carry no sequence, so this is not meaningful here
packet_rssi	Packet.RSSI : dBm, –127 to +20; 0x7F unavailable	Signal-to-noise proxy γ for the Mode-0 report’s Cramér–Rao bound
packet_antenna	Packet.Antenna : antenna identifier 0x01–0x04 used for the CS_SYNC	Calibration selection
measured_freq_offset	Measured.Freq.Offset : 15-bit signed, units 0.01 ppm, range ± 100 ppm; 0xC000 unavailable. Defined as $\text{FFO}[k]$ of [Core 6.3, Vol. 6, Part A, Sec. 3.5.1]	$\text{FFO}_{RX}[k]$. Initiator only

What the specification says this field *is*. Core 6.3 states that “the **Measured.Freq.Offset** parameter indicates the value of the FFO measured by the initiator device during the mode-0 CS step (see $\text{FFO}[k]$ in [Vol 6] Part A, Section 3.5.1)”. That is exactly the quantity the Mode-0 report derives, so the mapping is an identity rather than an interpretation: the controller has already performed the reconstruction from baseband beat to received frequency, applied its own local actuation error, and removed the peer’s actuation error from the table. What reaches you is ε in hundredths of a part per million.

Converting to the report’s dimensionless ε :

$$\varepsilon = \frac{\text{measured_freq_offset}}{100} \times 10^{-6}, \quad (1)$$

after sign-extending the 15-bit value and rejecting **0xC000**.

6 Mode-1 step data

```

struct bt_hci_le_cs_step_data_mode_1 {
    uint8_t packet_quality_bit_errors : 4;
    uint8_t packet_quality_aa_check : 4;
    uint8_t packet_nadm;
    uint8_t packet_rssi;
    union {
        int16_t toa_tod_initiator; /* valid when local role is initiator */
        int16_t tod_toa_reflector; /* valid when local role is reflector */
    };
    uint8_t packet_antenna;
} __packed;

```

Table 13: Mode-1 step fields. Fields shared with mode-0 are not repeated.

Zephyr field	Specification meaning	Use, and companion symbol
<code>packet_quality_bit_errors</code>	Payload bit errors, 0–15, meaningful when the RTT type carries a random or sounding sequence	Quality weight for the timing estimate
<code>packet_nadm</code>	Packet_NADM : 0x00 attack extremely unlikely through 0x06 extremely likely; 0xFF unknown, and the default for RTT types with no random or sounding sequence	NADM of the Mode-1 report. Integrity gate
<code>toa_tod_initiator</code>	ToA_ToD_Initiator : signed 16-bit, units 0.5 ns, known nominal offsets omitted ; 0x8000 unavailable	Yields A_I of the Mode-1 report once the nominal offsets are restored, Eq. (2)
<code>tod_toa_reflector</code>	ToD_ToA_Reflector : same encoding, reflector side	Yields B_R

6.1 The omitted nominal offsets

This is the single most consequential detail in the whole mapping, and the specification states it explicitly: the reported value is the time difference “where the known nominal offsets are omitted. The known offsets (i.e., the interlude time between packets and the length of the packet itself) are described in [Vol 6] Part H, Section 4.3.2 and [Vol 6] Part H, Section 4.3.4 for Mode-1 and Mode-3” [Core 6.3, Vol. 4, Part E].

The controller therefore reports a *residual* about the configured nominal step timing, not the absolute interval the Mode-1 report’s A_I and B_R denote. To recover them:

$$A_I = \underbrace{(T_{SY} + T_{RD} + T_{IP1})}_{\text{nominal, from the configuration}} + 0.5 \text{ ns} \times \text{toa_tod_initiator}, \quad (2)$$

$$B_R = (T_{SY} + T_{RD} + T_{IP1}) + 0.5 \text{ ns} \times \text{tod_toa_reflector}, \quad (3)$$

for a Mode-1 step, with the corresponding Mode-3 nominal from that report’s step structure. The round-trip time of the Mode-1 report then follows as $\text{RTT} = A_I - \tilde{B}_R - \hat{D}_{loop}$.

Why this closes an open question. The Mode-1 report’s host-interface appendix observed that a signed 16-bit field at 0.5 ns resolution spans only about $\pm 16.4 \mu\text{s}$, far narrower than the largest permitted interlude period, and concluded that the reported quantity “is therefore a differential referenced to the configured nominal step timing rather than an absolute interval”, to be confirmed against the specification. It is confirmed. The practical consequence is that **the nominal step timing is part of the measurement** — you cannot reconstruct a range from the step reports alone, and a stored procedure whose timing configuration was not stored with it is not reducible later.

6.2 The variant the documented structure does not show

Core 6.3 defines the mode-1 role-specific information to carry two additional fields when the conditions are right. For an initiator, “when the mode type is 1, the role is initiator, sounding phase-based ranging is supported, and the RTT type contains a sounding sequence”, the parameters are `Packet_Quality`, `Packet_NADM`, `Packet_RSSI`, `ToA_ToD_Initiator`, `Packet_Antenna`, `Packet_PCT1` and `Packet_PCT2` [Core 6.3, Vol. 4, Part E].

Table 14: The mode-1 sounding-sequence phase fields.

Field	Encoding	Meaning
<code>Packet_PCT1</code>	4 octets: bits 0–11 <i>I</i> (<code>sint12</code>), bits 12–23 <i>Q</i> (<code>sint12</code>), bits 24–31 reserved; 0xFFFFFFFF unavailable	Phase measured on the sounding sequence of the <code>CS_SYNC</code>
<code>Packet_PCT2</code>	Identical encoding	Second such term

This is phase-based ranging extracted from the RTT packet’s sounding sequence rather than from tones — the capability advertised as “CS phase-based ranging from RTT sounding sequence” and referred to in [Core 6.3, Vol. 6, Part A, Sec. 6.3] where the local actuation-error phase correction is applied over `T_SY_CENTER_DELTA`. The documented Zephyr `bt_hci_le_cs_step_data_mode_1` structure does not include these fields.

Do not size a mode-1 step from `sizeof()`. Because the presence of the two phase correction terms depends on role, on capability and on the configured RTT type, a mode-1 step record has more than one length. `step->data_len` is the authority. An application that casts blindly and advances by `sizeof(struct bt_hci_le_cs_step_data_mode_1)` will misparse every subsequent step in a configuration that carries the extra fields — and the Zephyr iterator does not protect you, because it advances by the length the controller reported, not by your struct size. Compare `data_len` against the expected length for your configuration and reject a mismatch rather than interpreting it.

7 Mode-2 step data

```
struct bt_hci_le_cs_step_data_tone_info {
    uint8_t phase_correction_term[3]; /* 12-bit I, 12-bit Q, packed */
    uint8_t extension_indicator : 4;
    uint8_t quality_indicator : 4;
} __packed;

struct bt_hci_le_cs_step_data_mode_2 {
    uint8_t antenna_permutation_index;
    struct bt_hci_le_cs_step_data_tone_info tone_info[]; /* N_AP + 1 entries */
} __packed;
```

The tone array is $N_{AP} + 1$ long, and the last entry is not a measurement. The extra slot is the tone extension slot. The Mode-2 report treats it as the step’s own noise and interference reference — valuable, and the only same-step, same-gain noise estimate available — but it is not an antenna path and it must not enter a phase-slope fit or a transform. Read `extension_indicator` rather than assuming the last index, because the specification describes it as indicating both *whether* a slot is an extension slot and whether a transmission was expected in it, and the reflector’s extension slot may legitimately carry no transmission at all.

Table 15: Mode-2 step fields.

Zephyr field	Specification meaning	Use, and companion symbol
<code>antenna_permutation_index</code>	Antenna_Path_Permutation_Index: which permutation of antenna paths this step used	Maps tone slot to physical antenna path. The Mode-2 report’s “silent and fatal” failure if mis-applied
<code>tone_info[k]</code>	Tone_PCT[k] , size $(N_{AP} + 1) \times 3$ octets	The array length is $N_{AP} + 1$, from the <i>header’s</i> num_antenna_paths
<code>.phase_correction_term</code>	Bits 0–11 <i>I</i> (sint12), 12–23 <i>Q</i> (sint12); with IPT enabled on the reflector, <i>Q</i> is set to 0	$PCT_{INIT}(f_k)$ or $PCT_{REFL}(f_k)$ of the Mode-2 report
<code>.quality_indicator</code>	Tone_Quality_Indicator[k] bits 0–3: tone quality per [Core 6.3, Vol. 6, Part H, Sec. 4.6]	TQI. Gating and weighting input
<code>.extension_indicator</code>	Tone_Quality_Indicator[k] bits 4–7: whether the slot is a tone extension slot and whether it was expected to carry a transmission	Identifies the extension slot, which must be excluded from any slope fit

7.1 Forming the two-way phase

The Mode-2 report’s central operation, restated in API terms. For channel k and antenna path p , with `ii` and `qq` the sign-extended halves from `bt_le_cs_parse_pct`:

$$H_2(f_k, p) = PCT_{INIT}(f_k, p) PCT_{REFL}(f_k, p), \quad \Theta(f_k, p) = \arg H_2(f_k, p). \quad (4)$$

Multiply the two complex numbers. Do not convert each to an angle and add: an isolated `atan2` result is wrapped, and the sum of two wrapped angles is not the wrapped sum.

Both devices’ terms are needed, and only one of them arrives locally. A subevent result delivers the *local* device’s phase correction terms. Equation (4) needs the peer’s as well, which must reach the ranging entity by some other path — unless Inline Phase Transfer is enabled, in which case the reflector has pre-rotated its carrier, its reported Q is zero by requirement, and the initiator’s own term already carries $2\theta_{CH}$. That is the whole subject of the companion IPT report, and it changes what you must store: with IPT the reflector’s term contributes amplitude only.

Converting a term’s magnitude to power, per [Core 6.3, Vol. 6, Part H, Sec. 4.6]:

$$P_{dBm} = 20 \log_{10} \left(\frac{\sqrt{I^2 + Q^2}}{2048} \right) + RPL_{dBm}, \quad (5)$$

with RPL the header’s `reference_power_level`, one value for the whole subevent.

8 Mode-3 step data

```

struct bt_hci_le_cs_step_data_mode_3 {
    uint8_t packet_quality_bit_errors : 4;
    uint8_t packet_quality_aa_check : 4;
    uint8_t packet_nadm;
    uint8_t packet_rssi;
    union {
        int16_t toa_tod_initiator;
        int16_t tod_toa_reflector;
    };
    uint8_t packet_antenna;
    uint8_t antenna_permutation_index;
    struct bt_hci_le_cs_step_data_tone_info tone_info[]; /* N_AP + 1 entries */
} __packed;

```

Mode-3 is the concatenation of the mode-1 and mode-2 field sets, and every field carries the meaning already given in Tables 13 and 15. The same `Packet_PCT1/Packet_PCT2` variant exists here as in mode-1, under the same conditions [Core 6.3, Vol. 4, Part E], and the same warning about `data_len` applies with more force because a mode-3 record has more optional content.

What mode-3 uniquely enables, in storage terms. Because the timing and phase fields arrive in one record, they are known to share a channel, an antenna path permutation, a gain state and an instant. That is the premise of the companion Mode-3 report's per-channel seeded unwrap and of its cross-observable consistency residual $\delta_k = \hat{d}_{\text{RTT},k} - \hat{d}_{\Theta,k}$. Preserve the pairing: store the timing and tone fields of a mode-3 step in one row, never in two tables joined later on channel, because a join on channel across a procedure silently reintroduces exactly the mismatch mode-3 exists to remove.

9 Master mapping

Table 16: Every result field, in all three vocabularies. “Report” cites the companion document in which the quantity is developed.

Zephyr	Core 6.3 HCI	Symbol	Report	Unavailable
<code>procedure_counter</code>	<code>Procedure_Counter</code>	—	—	—
<code>frequency_compensation</code>	<code>Frequency_Compensation</code>	FFO_{RX}^*	Mode-0	0xC000
<code>reference_power_level</code>	<code>Reference_Power_Level</code>	RPL	Mode-2	0x7F
<code>num_antenna_paths</code>	<code>Num_Antenna_Paths</code>	N_{AP}	Mode-2/3	0x00 = none
<code>num_steps_reported</code>	<code>Num_Steps_Reported</code>	M, K	all	—
<code>abort_step</code>	—	—	—	255 = none
<code>step->mode</code>	<code>Step_Mode</code>	—	all	—
<code>step->channel</code>	<code>Step_Channel</code>	f_k index	Mode-2/3	—
<code>packet_quality_aa_check</code>	<code>Packet_Quality</code> bits 0–3	—	all	—
<code>packet_quality_bit_errors</code>	<code>Packet_Quality</code> bits 4–7	—	Mode-1/3	0 ambiguous
<code>packet_nadm</code>	<code>Packet_NADM</code>	NADM	Mode-1/3	0xFF
<code>packet_rssi</code>	<code>Packet_RSSI</code>	γ proxy	all	0x7F
<code>packet_antenna</code>	<code>Packet_Antenna</code>	—	all	—
<code>measured_freq_offset</code>	<code>Measured_Freq_Offset</code>	$\text{FFO}_{RX}[k]$	Mode-0	0xC000
<code>toa_tod_initiator</code>	<code>ToA_ToD_Initiator</code>	A_I (residual)	Mode-1/3	0x8000
<code>tod_toa_reflector</code>	<code>ToD_ToA_Reflector</code>	B_R (residual)	Mode-1/3	0x8000
— (variant)	<code>Packet_PCT1/2</code>	—	§6.2	0xFFFFFFFF
<code>antenna_permutation_index</code>	<code>Antenna_Path_Permutation_Index</code>	—	Mode-2/3	—
<code>phase_correction_term</code>	<code>Tone_PCT[k]</code>	$\text{PCT}(f_k)$	Mode-2/3, IPT	—
<code>quality_indicator</code>	<code>Tone_Quality_Indicator</code> bits 0–3	TQI	Mode-2/3	unavailable code
<code>extension_indicator</code>	<code>Tone_Quality_Indicator</code> bits 4–7	—	Mode-2/3	—

10 What to store

One principle governs the schema: **persist what the controller reported, derive nothing irreversibly**. Every correction in the companion reports is one an implementation revises — the loop-delay calibration surface is refined, the actuation-error table is refreshed, the unwrap is re-seeded, the first-path threshold is re-characterised. A stored range is a dead end; a stored step

report can be re-reduced with next year’s calibration.

10.1 Configuration context, stored once per procedure

This is the layer most easily forgotten, and Section 6 shows why it is not optional: without the nominal step timing the reported timing residuals cannot be turned into intervals at all. Section 2 documents where each of these values comes from; the short version is that the ones that matter most are reported once and never again.

Table 17: Per-procedure context. None of this arrives in the subevent result; all of it is needed to reduce one. Section 2 documents each source, and Table 9 says which of them cannot be recovered after the fact.

Item	Source	Why it is needed
Local role (initiator / reflector)	<code>bt_conn_le_cs_config.role</code>	Decides which union member is valid and which mode-0 layout applies
<code>config_id</code> , <code>procedure_counter</code>	Header, plus the security session (§2.1)	Grouping keys
Main and sub mode, RTT type	<code>bt_conn_le_cs_config</code>	Decides step layout and whether the <code>Packet_PCT</code> variant is present
T_{IP1} , T_{IP2} , T_{FCS} , T_{PM}	<code>bt_conn_le_cs_config</code> , <i>negotiated</i> (§2.5)	Restoring the omitted nominal offsets, Eq. (2)
T_{SW}	Capability exchange, <i>not</i> the config event (§2.5)	Tone slot duration ($T_{SW} + T_{PM}$)($N_{AP} + 1$); antenna switching
T_{RD}	Fixed $5\mu s$	Nominal offset term
CS PHY	<code>bt_conn_le_cs_config.cs_sync_phy</code>	T_{SY} , and β_{rms} for the Mode-1 bound
Antenna configuration index, N_{AP}	<code>procedure_enable_complete</code> and header	Tone array length; per-path calibration
Channel map actually used	<code>bt_conn_le_cs_config.channel_map</code> , narrowed by classification	The aperture. The Mode-2 report shows the largest gap sets the unwrap limit
Scheduling cadence	<code>procedure_enable_complete</code> (§2.7)	The measurement time base; granted values differ from requested
IPT enabled	<code>cs.enhancements_1</code> bit 0	Whether the reflector’s Q is meaningful
Calibration versions: D_{loop} , $\Phi_{loop}(f)$, FAE table	Your own state; FAE from §2.9	Reproducibility of any later re-reduction

10.2 Per subevent

Every header field of Table 10, without exception, plus a local receive timestamp. The status and abort fields are small and are the only record of why a procedure was thin.

10.3 Per step

Store the step index (the iterator count), `mode`, `channel`, `data_len`, and the decoded fields for that mode. Three storage rules earn their place:

1. **Store raw integers, not converted physical values.** Keep `measured_freq_offset` as the signed count of 0.01 ppm and `toa_tod_initiator` as the signed count of 0.5 ns. Conversions are cheap to redo and impossible to undo, and a stored float has already lost the sentinel.
2. **Store sentinels as null, and store the fact separately.** Do not coerce 0x8000 or 0xC000 into a number. A boolean “available” column beside each such field makes every later query safe by construction.
3. **Keep the tone array ordered and complete**, including the extension slot, with its

`extension_indicator`. The Mode-2 report wants the extension slot as a noise reference; discarding it at ingest throws away the only same-step noise estimate.

Table 18: Minimum per-step record by mode. ✓ = store; — = not present.

Field	Mode-0	Mode-1	Mode-2	Mode-3
step index, mode, channel, <code>data_len</code>	✓	✓	✓	✓
<code>packet_quality_aa_check</code>	✓	✓	—	✓
<code>packet_quality_bit_errors</code>	✓	✓	—	✓
<code>packet_rssi</code>	✓	✓	—	✓
<code>packet_antenna</code>	✓	✓	—	✓
<code>measured_freq_offset</code> (initiator)	✓	—	—	—
<code>packet_nadm</code>	—	✓	—	✓
timing residual (<code>toa_tod/tod_toa</code>)	—	✓	—	✓
Packet_PCT1/2 if the variant is configured	—	✓	—	✓
<code>antenna_permutation_index</code>	—	—	✓	✓
per tone: I , Q , <code>quality_indicator</code> , <code>extension_indicator</code>	—	—	✓	✓

10.4 Derived values, stored alongside and never in place of the raw

Range estimates, ε , $\Theta(f_k)$, unwrap decisions and their seeds, gate outcomes and the calibration version used. The Mode-3 report’s consistency residual δ_k belongs here too. Storing the *decisions* as well as the results is what makes a later disagreement diagnosable.

11 How to use each report

11.1 Mode-0: establish the scale, then gate on it

Take `measured_freq_offset` from each mode-0 step whose `packet_quality_aa_check` is 0x0, convert with Eq. (1), and combine across the one to three mode-0 steps of the subevent. Compare the result against the header’s `frequency_compensation`, which is what the controller actually applied. The Mode-0 report’s checks apply directly: a spread across steps that correlates with channel indicates a stale or mis-indexed actuation-error table, and the specification’s own bound is $|\text{FFO}[k]| < 50 \text{ ppm}$ with $|\text{FFO}[k] - \text{FFO}[1]| < 1 \text{ ppm}$ across a subevent [Core 6.3, Vol. 6, Part A, Sec. 3.5.1] — two cheap ingest assertions.

The output feeds three consumers: rescaling B_R in the timing chain, bounding the drift term $c\varepsilon \Delta T/2$ in the phase chain, and, per the Mode-0 report’s downstream analysis, telling you whether Mode-0 is currently your limiting error source at all.

11.2 Mode-1: a coarse unambiguous range, and an integrity verdict

Restore the nominal offsets per Eq. (2), rescale the reflector interval by the Mode-0 result, subtract the calibrated loop delay, and halve. Gate on `packet_quality_aa_check`, on `packet_rssi`, and on `packet_nadm` — remembering that 0xFF is the *default* for RTT types without a random or sounding sequence and therefore means “not evaluated” rather than “suspicious”.

Use the result as the Mode-2 report says: as the seed that turns phase unwrapping from a sequential accumulation into independent nearest-integer decisions.

11.3 Mode-2: the reported range

Pair the local and peer phase correction terms per channel and antenna path, form H_2 by complex multiplication (Eq. (4)), apply the loop phase response, sort by frequency, exclude the extension slot and any tone whose `quality_indicator` is low or unavailable, and fit or transform. The Mode-2 report's aperture arithmetic then applies to whatever channel set survived gating — and since gating changes the largest gap, the safe unwrap limit is a property of the surviving set, not of the configured channel map.

11.4 Mode-3: use the pairing

Process the timing and phase halves as above, but per channel rather than per procedure. The Mode-3 report's two payoffs both require that the pairing survived storage: seed each channel's unwrap with that channel's own round-trip time, and compute δ_k as a per-step integrity and fault gate. A δ_k series that is offset but flat indicates calibration; one that grows with f_k indicates residual frequency; isolated outliers indicate interference on those channels.

12 Gaps and traps found while mapping

12.1 The packet-quality bitfield may be named opposite the specification's bit numbering

Core 6.3 defines `Packet_Quality` as one octet in which *bits 0 to 3* carry the access address check and *bits 4 to 7* carry the payload bit-error count [Core 6.3, Vol. 4, Part E]. The Zephyr structures declare, in this order:

```
uint8_t packet_quality_bit_errors : 4;
uint8_t packet_quality_aa_check : 4;
```

On the little-endian targets Zephyr predominantly runs on, the first-declared bitfield member occupies the least significant bits. Taken at face value that places `packet_quality_bit_errors` in bits 0–3 — which the specification says is the access address check — and `packet_quality_aa_check` in bits 4–7.

Verify this empirically before trusting either field. This report cannot resolve the question from documentation alone: bitfield allocation is implementation-defined in C, the field order was read from generated API documentation rather than from the source of the revision you are building, and the controller may or may not pack the octet the way the naming implies. What is certain is that the two interpretations are distinguishable by experiment and that the consequences differ sharply — an access-address value of `0x2` (“not found”) read as a bit-error count of 2 looks like a mildly degraded but usable packet.

The test is cheap. Force a link condition that produces access-address failures — attenuate deeply, or corrupt the access address — and observe which of the two members takes the values $\{0, 1, 2\}$ and which ranges over 0 to 15. The one saturating at 15 is the bit-error count. Record the answer for your Zephyr revision in the acceptance file.

12.2 The mode-1 and mode-3 phase variant is not expressed in the structures

Section 6.2 covers this. The practical rule is that `step->data_len` is the only reliable discriminator, and an application should compute the expected length from its own configuration and refuse to parse a step whose length disagrees.

12.3 The union does not tell you which member is valid

`toa_tod_initiator` and `tod_toa_reflector` are the same two octets. Nothing in the step record says which reading applies; it is your device’s role, which arrives from your own configuration. A reflector that reads `toa_tod_initiator` gets a plausible number with the wrong sign convention relative to the Mode-1 report’s A_I/B_R construction, and the resulting round-trip time is wrong without being absurd.

12.4 The reference sample covers only modes 1 and 2

Zephyr’s channel sounding sample switches on `BT_HCI_OP_LE_CS_MAIN_MODE_2` and `BT_HCI_OP_LE_CS_MAIN_MODE_1` and has no handling for modes 0 or 3. It is a good model for tone and timing extraction and should not be read as a model for a complete pipeline: the mode-0 frequency result and the mode-3 pairing — the two things the companion reports lean on hardest — are exactly what it omits.

12.5 The step timings are not in what you ask for

`bt_le_cs_create_config_params` has no timing fields, and `bt_conn_le_cs_config` — the structure handed back by the completion callback — has four: `t_ip1_time_us`, `t_ip2_time_us`, `t_fcs_time_us` and `t_pm_time_us`. The asymmetry is not an oversight in either the API or the specification: the Link Layers select these values between themselves [Core 6.3, Vol. 6, Part B, Table 5.2], and the Host is a reader, not a writer. The trap is that an application which mirrors its own `create_config_params` into its records — the natural thing to do, since that structure is the one it constructed — stores a configuration with no timings in it, and Eq. (2) then has no nominal to restore. Section 2.5 gives the full account.

12.6 T_{SW} is named in the specification’s prose but is not in the event

The LE CS Config Complete event’s description sentence lists `T_SW_Time` among the durations the event indicates; the event’s own parameter list stops at `T_PM_Time` [Core 6.3, Vol. 4, Part E, Sec. 7.7.65.42]. Zephyr follows the parameter list, so `bt_conn_le_cs_config` has no T_{SW} member. A reader who trusts the prose will look for a field that does not exist, and one who does not notice the omission will have no T_{SW} at all. It comes from the capability exchange instead — `t_sw_time`, or `t_sw_ipt_time_supported` when IPT is enabled — and it is required for the mode-2 tone geometry, where the tone occupies $(T_{SW} + T_{PM})(N_{AP} + 1)$.

12.7 Timing capability masks enumerate only the optional values

`t_ip1_times_supported` and its three siblings are masks over the *optional* shorter durations; the mandatory value of each parameter — $145\mu\text{s}$ for T_{IP1} and T_{IP2} , $150\mu\text{s}$ for T_{FCS} , $40\mu\text{s}$ for T_{PM} — sits at the top index and is supported by every conformant device [Core 6.3, Vol. 6, Part H, Tables 4.3, 4.5, 4.8, 4.9]. A mask of zero therefore reads as “no support” and means “mandatory timings only”. Any admission check written as “reject the peer if the mask is empty” rejects a perfectly usable device.

12.8 Requested scheduling is not granted scheduling

`min_procedure_interval`, `max_procedure_interval`, `min_subevent_len` and `max_subevent_len` are, in the specification’s own words, “recommendations to the Controller which it may ignore” [Core 6.3, Vol. 4, Part E, Sec. 7.8.140], and the procedure interval is ignored outright when `max_procedure_count` is 1. The values that will be used appear only in `bt_conn_le_cs_procedure_enable_complete` (Table 8). A campaign that records its requested cadence as the measurement time base — rather than the granted `procedure_interval` — will misdate every sample it takes, and the error

is systematic rather than noisy.

12.9 Sentinels in summary

Table 19: Values that are in range for their type but are not measurements. Every one must be tested before arithmetic.

Field	Sentinel	Consequence of using it as a number
frequency_compensation	0xC000	−163.84 ppm applied as a frequency correction
measured_freq_offset	0xC000	As above, per step
toa_tod / tod_toa	0x8000	−16.384 μ s of timing residual, some 2 500 m of range
reference_power_level	0x7F	Every tone power off by up to 107 dB
packet_rssi	0x7F	Gating and weighting corrupted
packet_nadm	0xFF	Reads as the worst possible attack verdict; is in fact “not evaluated”
Packet_PCT1/2	0xFFFFFFFF	Decodes to $I = Q = -1$; a valid-looking phase
quality_indicator	unavailable code	Tone included in the fit with no signal behind it
selected_tx_power (§2.7)	0x7F	Setup-time, not per step: a link budget off by up to 107 dB for the whole campaign
tx.power.delta (requested)	0x80	“No recommendation”, not −128 dB

13 Ingest checks worth asserting

1. Iterator count equals `header.num_steps_reported`.
2. If `subevent.done_status` is aborted, steps at index $\geq$ `abort_step` are discarded; if complete, `abort_step` is 255.
3. `step->data_len` equals the length expected for that mode, role and RTT type; reject rather than parse on mismatch (§6.2).
4. Mode-2 and mode-3 tone arrays contain exactly `header.num_antenna_paths+1` entries, and the entry flagged by `extension_indicator` is excluded from estimation.
5. Every sentinel of Table 19 is tested before conversion.
6. Mode-0 across the subevent: $|\text{FFO}[k]| < 50 \text{ ppm}$ and $|\text{FFO}[k] - \text{FFO}[1]| < 1 \text{ ppm}$ [Core 6.3, Vol. 6, Part A, Sec. 3.5.1].
7. `measured_freq_offset` present only in initiator-role mode-0 steps.
8. Sign convention: with a deliberately offset peer reference, the reported `measured_freq_offset` changes sign with the injected offset — the Mode-0 report’s s_{IQ} test, visible from the host.
9. Per-procedure context (Table 17) is stored before the first subevent is reduced; a procedure without it is not reducible later.
10. For mode-3, the timing and tone fields of one step are stored in one record.
11. A `bt_conn.le.cs_config` was latched for the `config_id` the subevent header names, and its four timing fields hold permitted values (Table 6); a zero there means the callback was missed, not that the timing is zero.
12. `header.num_antenna_paths` agrees with the N_{AP} implied by the granted `tone_antenna_config_selection` of Table 8; a mismatch means the stored context belongs to a different procedure.
13. `header.config_id` matches the configuration in force, and the record carries the security session as well as `procedure_counter` (§2.1).

14. Reduction uses the *granted* cadence from `procedure_enable_complete`, never the requested `min/max_procedure_interval`.
15. Per-path corrections index through `bt_le_cs_get_antenna_path`, not through the raw tone index, whenever `antenna_permutation_index` is non-zero.

14 Conclusions

The Zephyr API is a thin, faithful wrapper over the host interface, and almost all of the interpretive burden it leaves behind is inherited from that interface rather than introduced by it. Three properties account for most of the difficulty. The step payload is untyped and its layout depends on mode, role and configuration, so the application must know things the data does not carry. Several fields use in-range sentinel values that are silently catastrophic if treated as measurements. And the timing fields report residuals about a nominal step timing that the result does not include, which makes the procedure's configuration part of the measurement rather than context for it.

The configuration path of Section 2 sharpens that last point into a design constraint. The Host does not set a CS procedure so much as bound one: the step timings are chosen by the two Link Layers and reported once, the scheduling parameters are explicitly advisory and answered only in the procedure-enable completion, and T_{SW} is not in the configuration event at all. Each of those values is delivered to exactly one callback and is unrecoverable afterwards, which turns an apparently minor design decision — which callbacks latch state — into the thing that determines whether a dataset can be re-reduced later. Table 9 is the short form of that requirement.

The storage recommendation follows from the last of those and from the character of the companion reports' corrections: keep the controller's raw integers, keep the configuration that gives them meaning, keep the gating decisions, and treat any derived range as a cache. Every correction those reports describe is one that gets revised, and a stored step report can be re-reduced while a stored metre value cannot.

Three items should be resolved locally before this mapping is relied on in code: the packet-quality bitfield orientation of Section 12, which is an experiment of a few minutes; whether your configuration produces the mode-1 and mode-3 variant carrying `Packet_PCT1` and `Packet_PCT2`, which determines every step length in the buffer; and the timing values your controller pair actually negotiates, which are observable in one print statement from the configuration callback and are otherwise guesswork.

A Enumerated values

Table 20: Values from [Core 6.3, Vol. 4, Part E] and [Core 6.3, Vol. 6, Part H, Sec. 4.6]. Zephyr symbol names are given where observed in the public API; numeric equality with the specification values should be confirmed against your revision.

Quantity	Value	Meaning
Packet_Quality bits 0–3	0x0	Access address check successful, all bits match
	0x1	One or more bit errors
	0x2	Access address not found
Packet_Quality bits 4–7	0–15	Payload bit errors; 0 may mean none <i>or</i> no report; 15 means 15 or more
Packet_NADM	0x00–0x06	Attack extremely unlikely → extremely likely
	0xFF	Unknown; the default for RTT types without a random or sounding sequence
Tone quality	bits 0–3 of Tone_Quality_Indicator	High / medium / low / unavailable; Zephyr exposes BT_HCI_LE_CS_TONE_QUALITY_LOW and ...UNAVAILABLE
Tone extension	bits 4–7	Whether the slot is a tone extension slot, and whether a transmission was expected
Packet_RSSI	–127 to +20 dBm	0x7F unavailable
Packet_Antenna	0x01–0x04	Antenna identifier for the CS_SYNC
Num_Antenna_Paths	0x00	No phase measurement in the step
	0x01–0x04	N_{AP}
Procedure done	complete / partial / aborted	bt_conn_le_cs_procedure_done_status
Subevent done	complete / aborted	bt_conn_le_cs_subevent_done_status
Procedure abort	no abort, local or remote request, too few channels, channel-map instant passed, unspecified	bt_conn_le_cs_procedure_abort_reason
Subevent abort	no abort, local or remote request, no CS_SYNC received, scheduling conflict, unspecified	bt_conn_le_cs_subevent_abort_reason

References

- [1] Zephyr Project, headers `cs.h`, `conn.h` and `hci_types.h` under `include/zephyr/bluetooth/`, main branch.
<https://github.com/zephyrproject-rtos/zephyr>
- [2] Zephyr Project, API documentation, Channel Sounding (CS) group and the `bt_hci_le_cs_step_data_*` structure references.
https://docs.zephyrproject.org/apidoc/latest/group__bt__le__cs.html (Channel Sounding group)
- [3] Zephyr Project, `samples/bluetooth/channel_sounding`, including `src/distance_estimation.c`.
https://github.com/zephyrproject-rtos/zephyr/tree/main/samples/bluetooth/channel_sounding
- [4] Bluetooth SIG, *Bluetooth Core Specification, Version 6.3*, Vol. 4, Part E, “Host Controller Interface Functional Specification,” LE CS Subevent Result event and the `ModeRoleSpecific_Info` parameter definitions, adopted 2026.
- [5] Bluetooth SIG, *Bluetooth Core Specification, Version 6.3*, Vol. 6, Part A, Secs. 3.5 and 6.2–6.5, adopted 2026.
- [6] Bluetooth SIG, *Bluetooth Core Specification, Version 6.3*, Vol. 6, Part H, Secs. 4.3.1–4.3.4 and 4.6, adopted 2026.

- [7] This series: *Bluetooth CS Mode-0 Frequency and Timing Calibration*, Rev. 1.0; *Mode-1 RTT Estimation*, Rev. 1.0; *Mode-2 Phase-Based Ranging*, Rev. 1.0; *Mode-3 Combined RTT and PBR*, Rev. 1.0; *Inline Phase Transfer*, Rev. 1.0.